Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-213БВ-25

Студент: Крайнова О.А.

Преподаватель: Бахарев В.Д.

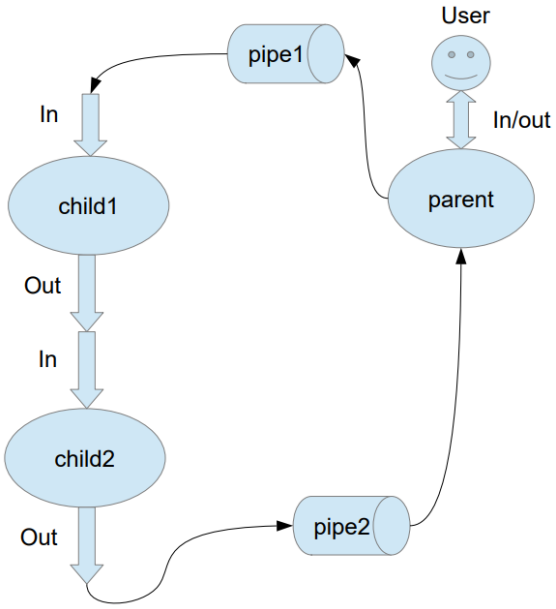
Оценка: _____

Дата: 14.10.26

Москва, 2026

Постановка задачи

Вариант 13.



Родительский процесс создает два дочерних процесса. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Child1 и Child2 можно «соединить» между собой дополнительным каналом. Родительский и дочерний процесс должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Child1 переводит строки в нижний регистр. Child2 превращает все пробельные символы в символ «_». Child2 пересылает результат своей работы родительскому процессу. Родительский процесс полученный результат выводит в стандартный поток вывода.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- int pipe(int pipefd[2]); – создание неименованного канала для передачи данных между процессами.
- int dup2(int oldfd, int newfd); – переназначение файлового дескриптора.
- int execl(const char *path, const char *arg, ...); – замена образа памяти процесса.
- void _exit(int status); – завершение выполнения процесса и возвращение статуса.
- int close(int fd); – закрытие файлового дескриптора.
- pid_t waitpid(pid_t pid, int *status, int options); – ожидание завершения дочернего процесса.
- ssize_t write(int fd, const void *buf, size_t n); – запись в файловый дескриптор.
- ssize_t read(int fd, void *buf, size_t nbytes); – чтение из файлового дескриптора.

Система состоит из родительского и двух дочерних процессов через fork(), которые обмениваются данными через три последовательно соединенных канала: pipe1, pipe2 и pipe_mid. Родительский процесс считывает строки от пользователя с клавиатуры и отправляет их в первый канал pipe1. В каждом дочернем процессе с помощью dup2() стандартный ввод и вывод перенаправляются на соответствующие концы каналов, после чего через execl() запускается программа. Процесс child1 переводит текст в нижний регистр, затем он (процесс child1) отправляет

результат через промежуточный канал pipe_mid ко второму дочернему процессу child2, который заменяет пробельные символы на подчеркивания '_'. Родительский процесс закрывает неиспользуемые концы каналов, считывает итоговый результат из последнего канала pipe2, выводит его на экран и дожидается завершения дочерних процессов через waitpid().

Код программы

child1.c

```
#include <unistd.h>

#include <ctype.h>

int main(){
    char buffer[1024];
    ssize_t bytes_read;

    while ((bytes_read = read(0, buffer, sizeof(buffer))) > 0) {
        for (ssize_t i = 0; i < bytes_read; i++) {
            buffer[i] = tolower((unsigned char)buffer[i]);
        }

        if (write(1, buffer, bytes_read) < 0){
            write(2, "write error\n", 12);
            return 1;
        }
    }

    if (bytes_read < 0) {
        write(2, "read error\n", 11);
        return 1;
    }

    return 0;
}
```

child2.c

```
#include <unistd.h>
#include <ctype.h>

int main(){
    char buffer[1024];
    ssize_t bytes_read;

    while ((bytes_read = read(0, buffer, sizeof(buffer))) > 0) {
        for (ssize_t i = 0; i < bytes_read; i++) {
            if (isspace((unsigned char)buffer[i]))
                buffer[i] = '_';
        }
```

```
        if (write(1, buffer, bytes_read) < 0){
            write(2, "write error\n", 12);
            return 1;
        }
    }

    if (bytes_read < 0) {
        write(2, "read error\n", 11);
        return 1;
    }

    return 0;
}
```

parent.c

```
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipe1[2];
    int pipe2[2];
    int pipe_mid[2];

    if (pipe(pipe1) < 0 || pipe(pipe2) < 0 || pipe(pipe_mid) < 0) {
        write(2, "pipe error\n", 11);
        return 1;
    }

    pid_t child1_pid = fork();
    if (child1_pid < 0) {
        write(2, "fork error\n", 11);
        return 1;
    }

    if (child1_pid == 0){
        if (dup2(pipe1[0], 0) < 0 || dup2(pipe_mid[1], 1) < 0) {
            write(2, "dup2 error\n", 11);
            _exit(1);
        }

        close(pipe1[0]); close(pipe1[1]);
        close(pipe2[0]); close(pipe2[1]);
        close(pipe_mid[0]); close(pipe_mid[1]);

        execl("./child1", "child1", NULL);
        write(2, "execl error\n", 12);
        _exit(1);
```

```
}


pid_t child2_pid = fork();
if (child2_pid < 0) {
    write(2, "fork error\n", 11);
    return 1;
}

if (child2_pid == 0){
    if (dup2(pipe_mid[0], 0) < 0 || dup2(pipe2[1], 1) < 0) {
        write(2, "dup2 error\n", 11);
        _exit(1);
    }

    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    close(pipe_mid[0]); close(pipe_mid[1]);

    execl("./child2", "child2", NULL);
    write(2, "execl error\n", 12);
    _exit(1);
}

close(pipe1[0]);
close(pipe2[1]);
close(pipe_mid[0]);
close(pipe_mid[1]);


char inbuf[1024];
char outbuf[1024];
ssize_t bytes_read;
ssize_t bytes_write;

while((bytes_read = read(0, inbuf, sizeof(inbuf))) > 0) {
    bytes_write = write(pipe1[1], inbuf, bytes_read);
    if (bytes_write < 0) {
        write(2, "write error\n", 12);
        return 1;
    }

    ssize_t out_read = 0;
    while(out_read < bytes_read) {
        ssize_t n = read(pipe2[0], outbuf + out_read, bytes_read - out_read);
        if (n < 0) {
            write(2, "read error\n", 11);
            return 1;
```

```
            }

            if (n == 0) break;
            out_read += n;
        }

        if (write(1, outbuf, out_read) < 0) {
            write(2, "write error\n", 12);
            return 1;
        }

        if (out_read == 0 || outbuf[out_read - 1] != '\n')
            write(1, "\n", 1);
    }

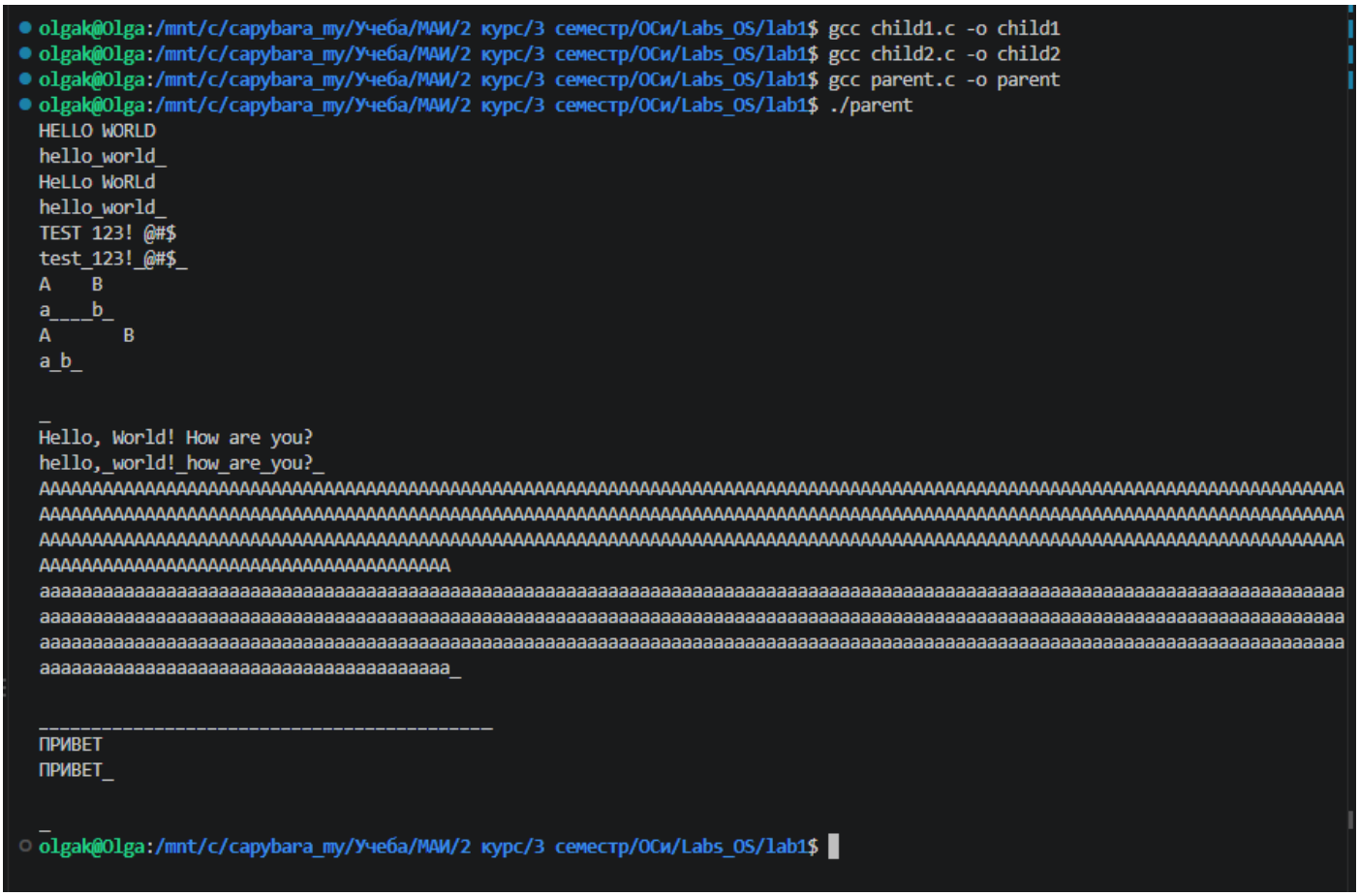
    if (bytes_read < 0) {
        write(2, "read error\n", 11);
        return 1;
    }

    close(pipe1[1]);
    close(pipe2[0]);

    int status;
    waitpid(child1_pid, &status, 0);
    waitpid(child2_pid, &status, 0);

    return 0;
}
```

Протокол работы программы


olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab1$ gcc child1.c -o child1
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab1$ gcc child2.c -o child2
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab1$ gcc parent.c -o parent
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab1$ ./parent
HELLO WORLD
hello_world_
HeLLo WoRLd
hello_world_
TEST 123! @#$
test_123!_@#$_
A B
a___b_
A B
a_b_
_
Hello, World! How are you?
hello,_world!_how_are_you?_
AAAA